

7. Artificial Neural Networks

Machine learning

Jiri Chmelik

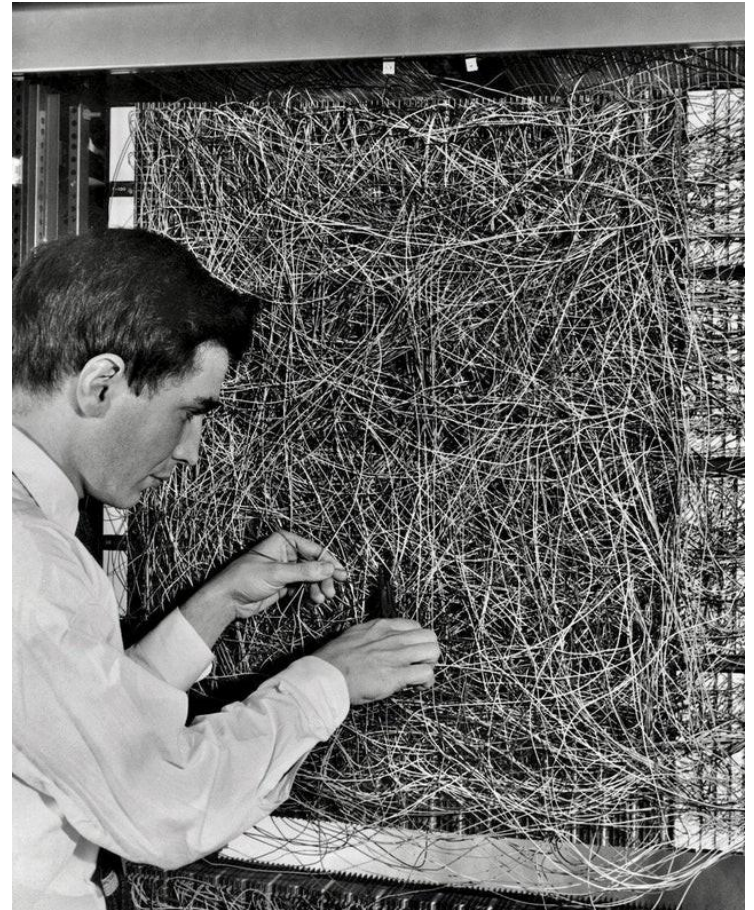
Department of Biomedical Engineering

Introduction

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization

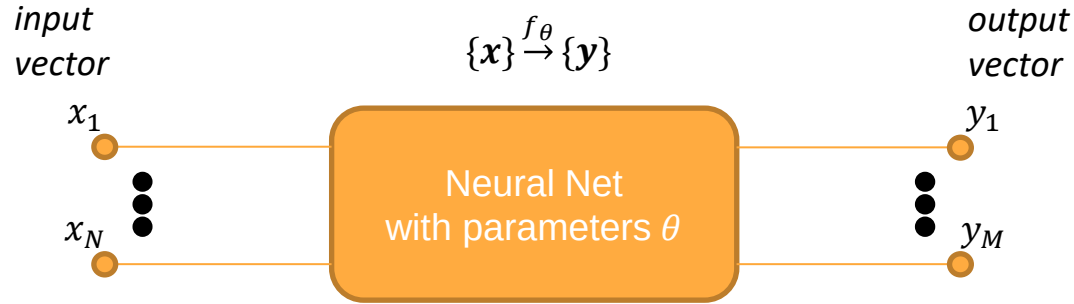
Basics

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization



Frank Rosenblatt and his Mark I Perceptron

Principle of Neural Nets



$$\hat{y} = f_\theta(x)$$

Problem: find optimal parameters θ

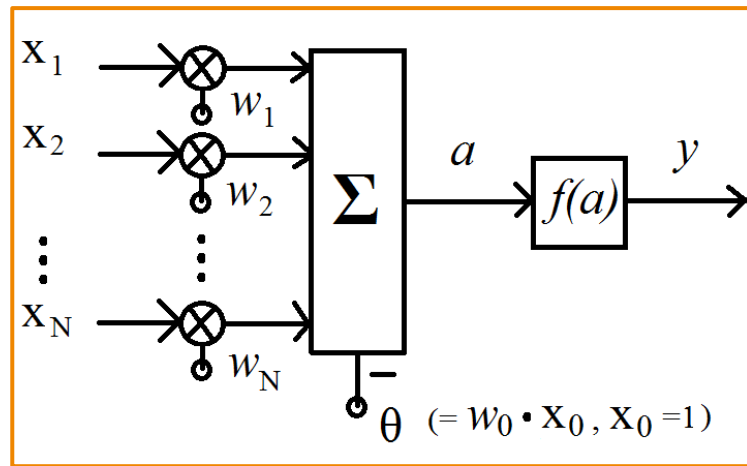
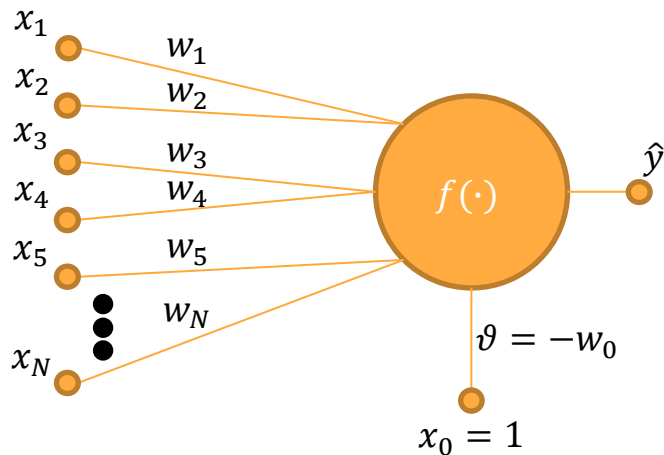
$$\underset{\theta}{\text{minimize}} \quad L(y, f_\theta(x)) \quad L - \text{error/loss function}$$

Solution: Gradient descent

$$\theta^{it+1} = \theta^{it} - \mu \nabla_\theta L(y, f_\theta(x)) \quad \mu - \text{learning rate/step}$$

Artificial neuron

Artificial Neural Networks



Output:

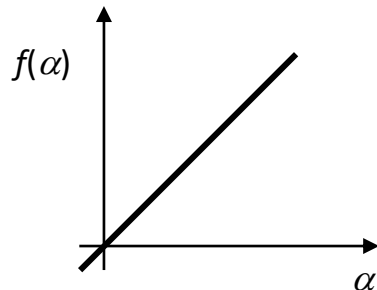
$$y = f \left[\sum_{i=1}^N w_i x_i - v \right] = f \left[\sum_{i=0}^N w_i x_i \right], \quad \text{where } x_0 = 1, w_0 = -v$$

Labels for the equation components:

- activation function: points to f
- inner potential: points to the first summation $\sum_{i=1}^N w_i x_i - v$
- threshold: points to $-v$
- weights: points to w_i
- input: points to x_i

Activation functions

- **Linear**



$$f(\alpha) = \alpha \quad f(\alpha) \in \mathbb{R}$$

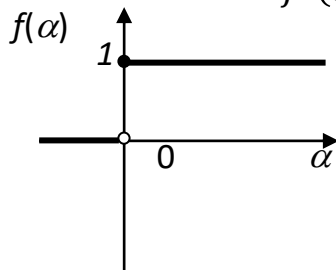
$$f'(\alpha) = 1$$

- **Step functions**

Heaviside step function (unit step function):

$$f(\alpha) = \frac{\text{sign}(\alpha) + 1}{2} \quad f(\alpha) \in \{0, 1\}$$

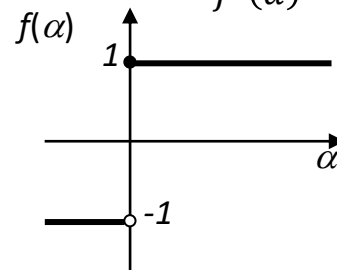
$$f'(\alpha) = \delta(\alpha)$$



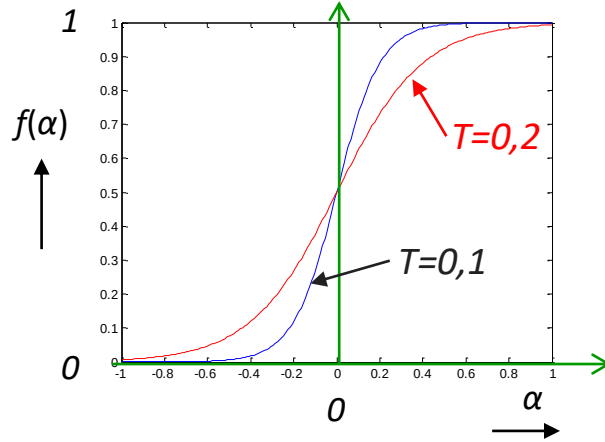
Signum:

$$f(\alpha) = \text{sign}(\alpha) \quad f(\alpha) \in \{-1, 1\}$$

$$f'(\alpha) = \delta(\alpha)$$



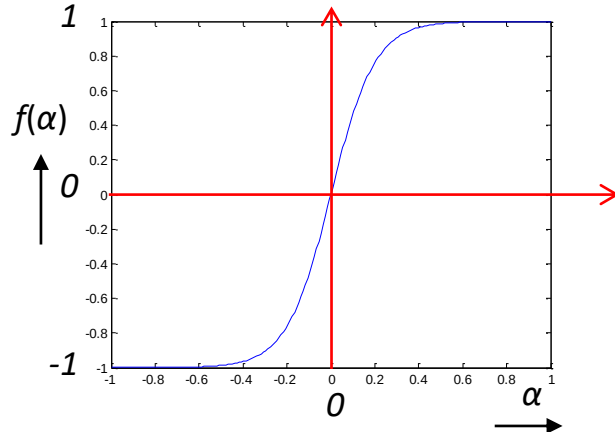
• Sigmoid



$$\sigma(\alpha) = f(\alpha) = \frac{1}{1 + e^{-\frac{\alpha}{T}}} \quad f(\alpha) \in [0, 1]$$

$$f'(\alpha) = f(\alpha)(1 - f(\alpha))$$

• Hyperbolic tangent (tanh)



$$f(\alpha) = 2\sigma(2\alpha) - 1 = 2 \frac{1}{1 + e^{-2\alpha}} - 1 = \frac{2}{1 + e^{2\alpha}} - 1$$

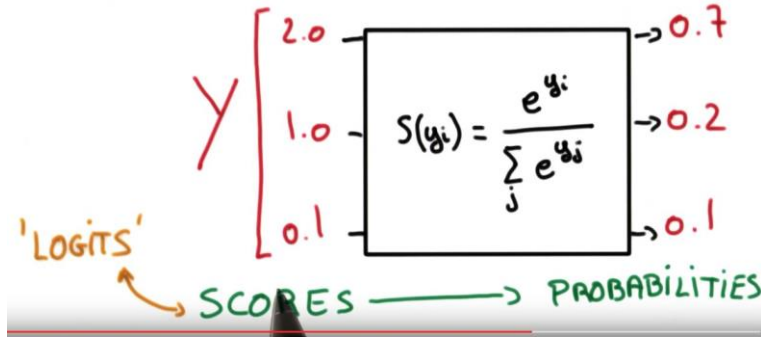
$$= \frac{e^{2\alpha} - 1}{e^{2\alpha} + 1} \quad f(\alpha) \in [-1, 1]$$

$$f'(\alpha) = 4f(\alpha)(1 - f(\alpha))$$

- **Softmax**

Artificial Neural Networks

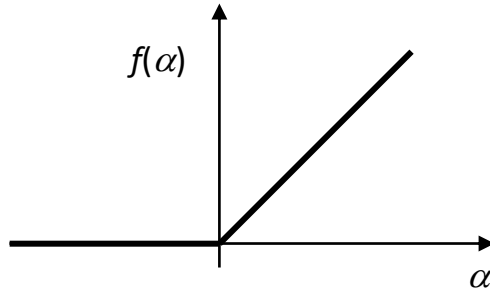
SOFTMAX



$$f(\alpha_i) = \frac{e^{\alpha_i}}{\sum_{j=1}^K e^{\alpha_j}} \quad f(\alpha_i) \in [0, 1], \sum_{j=1}^K f(\alpha_j) = 1$$

$$f'(\alpha_i) = \begin{cases} f(\alpha_i)(1 - f(\alpha_i)) & \text{if } i = j \\ -f(\alpha_i)f(\alpha_j) & \text{if } i \neq j \end{cases}$$

- **ReLU (Rectified Linear Unit)**

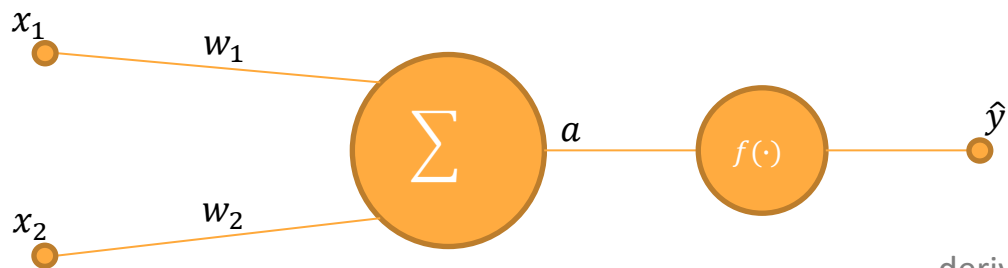


$$f(\alpha) = \max(0, \alpha) \quad f(\alpha) \in [0, \infty)$$

$$f'(\alpha) = \begin{cases} 0 & \text{if } \alpha < 0 \\ 1 & \text{if } \alpha > 0 \end{cases}$$

Derivation of single neuron

Artificial Neural Networks



derivation of loss function
(for $f(\cdot)$ – linear activation function)

$$L = \frac{1}{2} (x_1 w_1 + x_2 w_2 - y)^2$$

$$\frac{dL}{dw_i} = 2 \frac{1}{2} (x_1 w_1 + x_2 w_2 - y) x_i$$

generalized derivation for any loss function

$$\frac{dL}{dw_i} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{da}{dw_i}$$

$$D = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$$

loss function

$$L = \frac{1}{2} (\hat{y} - y)^2$$

derivation of loss function
(for $f(\cdot)$ – sigmoid activation function)

$$L = \frac{1}{2} \left(\frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2)}} - y \right)^2$$

$$\frac{dL}{dw_i} = 2 \frac{1}{2} [f(x_1 w_1 + x_2 w_2) (1 - f(x_1 w_1 + x_2 w_2)) - y] x_i$$

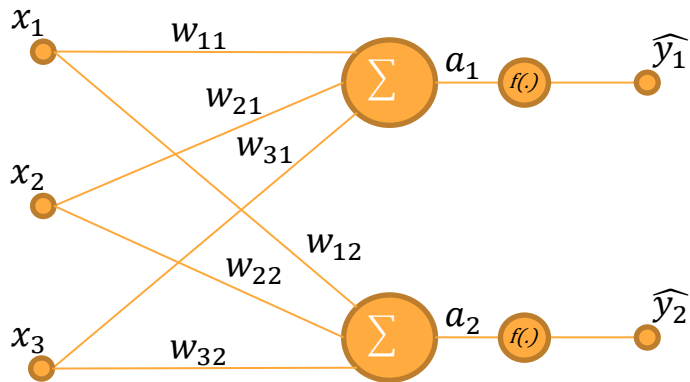
$$\frac{dL}{d\hat{y}} = 2 \frac{1}{2} (\hat{y} - y) \cdot 1 = (\hat{y} - y) \quad \text{loss fcn. dependent}$$

$$\frac{d\hat{y}}{da} \rightarrow \text{dependent on used activation fcn.}$$

$$\frac{da}{dw_i} = x_i$$

Matrix syntax

Artificial Neural Networks



$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\mathbf{a} = \mathbf{W} \mathbf{x}$$

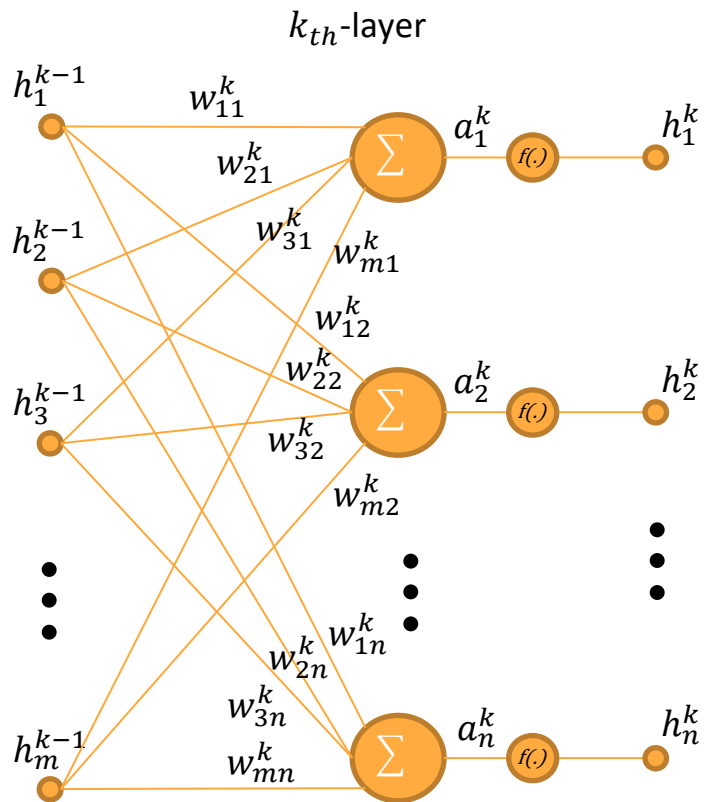
$$\hat{\mathbf{y}} = f(\mathbf{a}) = \begin{bmatrix} f(a_1) \\ f(a_2) \end{bmatrix}$$

loss function

$$L = \frac{1}{2} (\hat{\mathbf{y}} - \mathbf{y})^2 = \frac{1}{2} \left[(f(a_1) - y_1)^2 + (f(a_2) - y_2)^2 \right]$$

$$\nabla_{\mathbf{W}} L = \left(\nabla_{\hat{\mathbf{y}}} L \odot f'(\mathbf{a}) \right) \mathbf{x}^T = \left(\begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \end{bmatrix} \odot \begin{bmatrix} f'(a_1) \\ f'(a_2) \end{bmatrix} \right) \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = \begin{bmatrix} \nabla_{w_{11}} L_1 & \nabla_{w_{21}} L_1 & \nabla_{w_{31}} L_1 \\ \nabla_{w_{12}} L_2 & \nabla_{w_{22}} L_2 & \nabla_{w_{32}} L_2 \end{bmatrix}$$

Derivation of general neural net – each layer + NN learning



- define:

$$\mathbf{h}_0 = \mathbf{x}$$

- compute forward propagation:

for $k = 1$ to K

$$\mathbf{a}_k = \mathbf{W}_k \mathbf{h}_{k-1}$$

$$\mathbf{h}_k = f(\mathbf{a}_k)$$

note:

$$\mathbf{h}_K = \hat{\mathbf{y}}$$

$$\nabla_{\mathbf{h}_K} L = \nabla_{\hat{\mathbf{y}}} L$$

- compute gradient backpropagation:

for $k = K$ to 1

$$\nabla_{\mathbf{a}_k} L = \nabla_{\mathbf{h}_k} L \odot f'(\mathbf{a}_k)$$

$$\nabla_{\mathbf{h}_{k-1}} L = \mathbf{W}_k^T \nabla_{\mathbf{a}_k} L$$

$$\nabla_{\mathbf{W}_k} L = \nabla_{\mathbf{a}_k} L \mathbf{h}_{k-1}^T$$

- update weights (by vanilla SGD):

$$\mathbf{W}^{(it+1)} = \mathbf{W}^{(it)} - \mu \nabla_{\mathbf{W}^{(it)}} L$$

μ – learning rate

it – iteration

Learning Algorithms

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization

Learning Algorithms – “Vanilla” GD

1. Gradient Descent (GD)

- weight update after each epoch (one loss and gradient value for whole training dataset)
- not used – slow, unfeasible for large training dataset, local extrema of the loss function are lost
- for appropriate learning rate GD converges to the global minimum for convex loss functions, to the local minimum for non-convex loss functions
- “online” update is not allowed

$$\mathbf{W}^{(epoch+1)} = \mathbf{W}^{(epoch)} - \mu \nabla_{\mathbf{W}^{(epoch)}} L(\mathbf{x}, \mathbf{y}, \mathbf{W}^{(epoch)})$$

2. Stochastic Gradient Descent (GD)

- weight update after each iteration (one loss and gradient value for each sample of training dataset) – computational demanding, but faster than full GD
- high variance of updates – fluctuation
- enables jumping to better local extrema
- overshooting the global extreme – can be solved by slow decreasing the learning rate
- shuffle the training data randomly at the beginning of each epoch

$$\mathbf{W}^{(it+1)} = \mathbf{W}^{(it)} - \mu \nabla_{\mathbf{W}^{(it)}} L(\mathbf{x}^{(it)}, \mathbf{y}^{(it)}, \mathbf{W}^{(it)})$$

Learning Algorithms – “Vanilla” GD

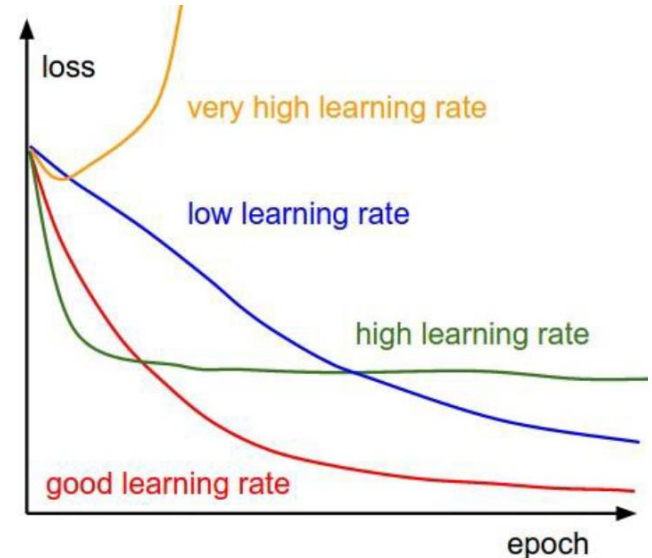
3. Mini-batch Gradient Descent

- “between” GD and SGD
- weight update is done after few (n) iterations
- reduces the variance of weight updates
- enables efficient parallelization
- n is in the range of tens to hundreds/thousands – depends on solved problem, size of training data, and available memory
- commonly used algorithm

$$\mathbf{W}^{(it+1)} = \mathbf{W}^{(it)} - \mu \nabla_{\mathbf{W}^{(it)}} L(\mathbf{x}^{(it:it+n)}, \mathbf{y}^{(it:it+n)}, \mathbf{W}^{(it)})$$

Learning Algorithms – “Vanilla” GD – problems

- chose a proper learning rate?
- learning rate schedules...
- same learning rate for all parameters?
- problem of saddle point plateau – same error in all directions -> zero gradient...
- problem with local extrema



Learning Algorithms – GD modified

4. SGD with Momentum

- accelerates SGD movement in the same direction as was in the previous iteration
- reduces SGD movement in the opposite direction
- faster convergence and reduced variance of weight updates than “vanilla” SGD
- additional parameter α
- better trails the slope of loss function than “vanilla” SGD
- may solve the problem with stuck in local extrema
- exponentially decayed average of the past gradients

$$\mathbf{v}^{(it+1)} = \alpha \mathbf{v}^{(it)} + \mu \nabla_{\mathbf{w}^{(it)}} L$$

$$\mathbf{w}^{(it+1)} = \mathbf{w}^{(it)} - \mathbf{v}^{(it+1)}$$

$\mathbf{v}$ – velocity

α – parameter of momentum – $[0,1]$ – 0.9

Learning Algorithms – GD modified

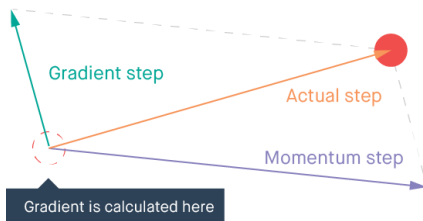
5. Nesterov Accelerated Gradient (NAG)

- similar as SGD with momentum, but:
- firstly estimates the movement in actual iteration – $\mathbf{W}^{(it)} - \alpha \mathbf{v}^{(it)}$
- then computes gradient of error in the estimated position and corrects the first estimation
- and then computes final update of weights
- prevents the fast movement in incorrect direction – faster convergence than SGD with Momentum in many applications – “smarter” modification of size and direction velocity

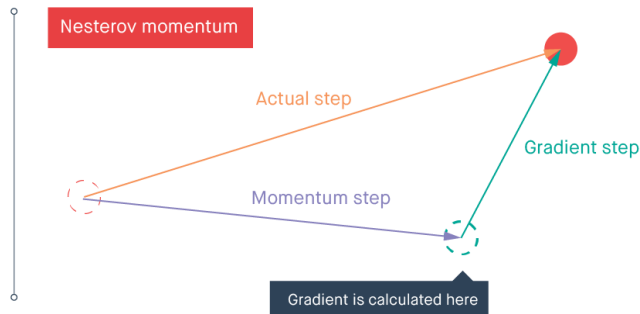
$$\mathbf{v}^{(it+1)} = \alpha \mathbf{v}^{(it)} + \mu \nabla_{\mathbf{W}^{(it)}} L(\mathbf{W}^{(it)} - \alpha \mathbf{v}^{(it)})$$

$$\mathbf{W}^{(it+1)} = \mathbf{W}^{(it)} - \mathbf{v}^{(it+1)}$$

Standard momentum



Nesterov momentum



Learning Algorithms – GD modified – problems

- partly solves the problem with zero gradient
- partly solves the problem with local extrema
- additional hyperparameter (α)
- chose a proper learning rate?
- same learning rate for all parameters?
- learning rate schedules...

Learning Algorithms – GD with learning rate modification

6. Adagrad

- during learning changes LR for each parameter (weight) separately
- based on sum of squares of the past accumulated gradients for each weight – matrix $\mathbf{G}_{it}$ with smart size
- no need of manual tune of LR
- accumulation of gradient – too aggressive shrinking of the LR during training -> no change of weights

$$\mathbf{g}^{(it+1)} = \nabla_{\mathbf{w}^{(it)}} L$$
$$\mathbf{w}^{(it+1)} = \mathbf{w}^{(it)} - \frac{\mu}{\sqrt{\mathbf{G}_{it} + \epsilon}} \odot \mathbf{g}^{(it+1)} \quad \epsilon - \text{smoothing term}$$

7. Adadelta/RMSprop

- extension of Adagrad
- takes only w of past accumulated gradients instead of all past squared gradient as exponentially decayed average
- solves shrinking problem of Adagrad

Learning Algorithms – GD with learning rate modification

8. Adaptive Moment Estimation (Adam)

- combines exponentially decaying average of past squared gradients $\mathbf{v}_{it}$ (such as Adadelta/RMSprop) and exponentially decaying average of past gradients $\mathbf{m}_{it}$ (such as momentum)
- momentum -> behaves like a friction in Adam
- nowadays, the most frequently used algorithm

$$\mathbf{m}^{(it+1)} = \beta_1 \mathbf{m}^{(it)} + (1 - \beta_1) \nabla_{\mathbf{w}^{(it)}} \mathbf{L}$$

$$\mathbf{v}^{(it+1)} = \beta_2 \mathbf{v}^{(it)} + (1 - \beta_2) (\nabla_{\mathbf{w}^{(it)}} \mathbf{L})^2$$

β_1 – decay rate of the first moment $[0,1] - 0.9$

β_2 – decay rate of the second moment $[0,1] - 0.999$

ϵ – smoothing term – arbitrary small – 10^{-8}

$$\mathbf{w}^{(it+1)} = \mathbf{w}^{(it)} - \frac{\mu}{\sqrt{\mathbf{v}^{(it)} + \epsilon}} \odot \mathbf{m}^{(it)}$$

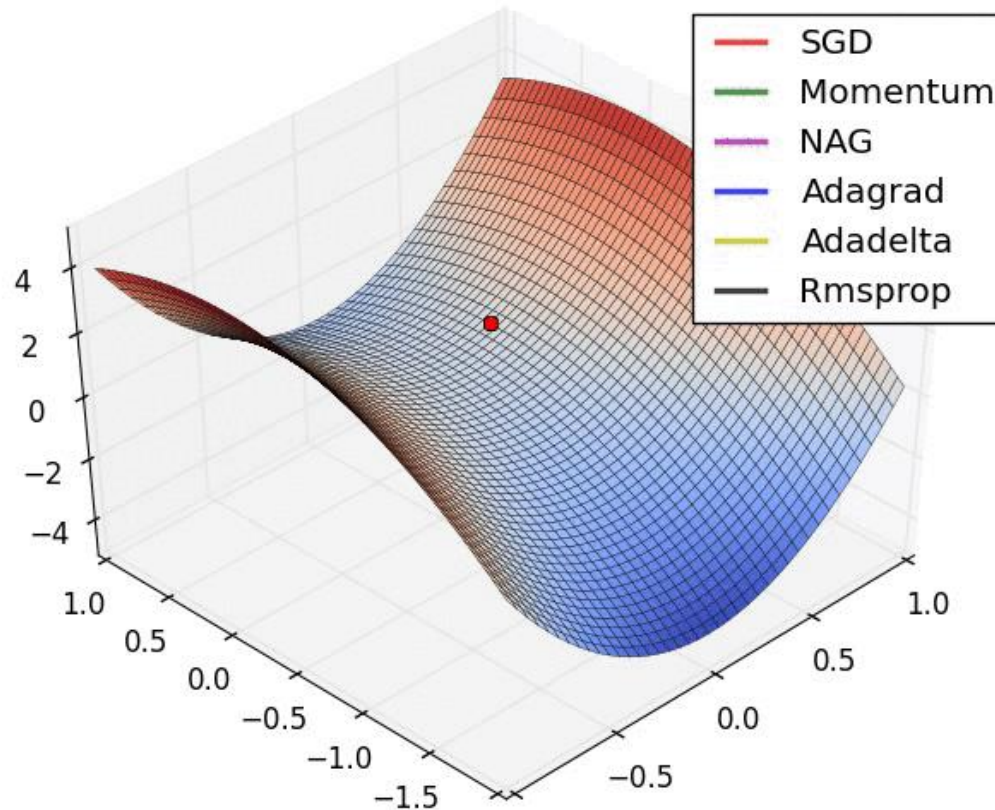
9. Nesterov-accelerated Moment Estimation (Nadam)

- momentum -> changed for Nesterov momentum

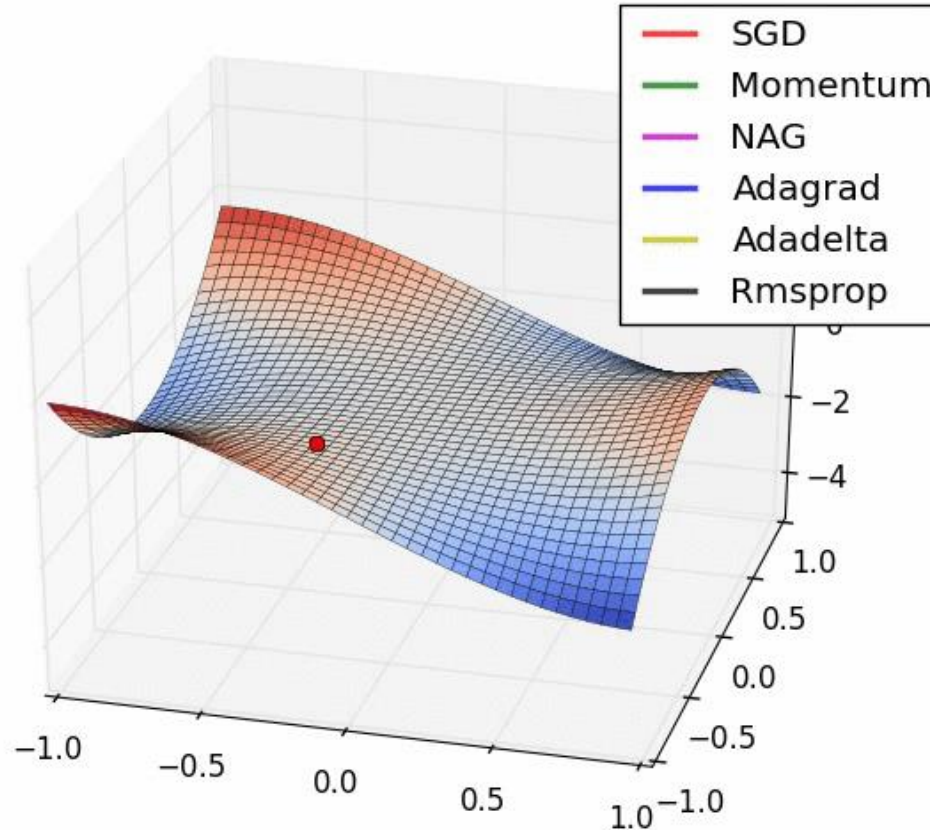
Learning Algorithms – GD with learning rate modification

- solves the problem with zero gradient
- solves the problem with local extrema
- solves the problem with initial learning rate
- solves the problem with learning rate for each parameter (weight)
- more additional hyperparameters (β_1, β_2)
- learning rate schedules...

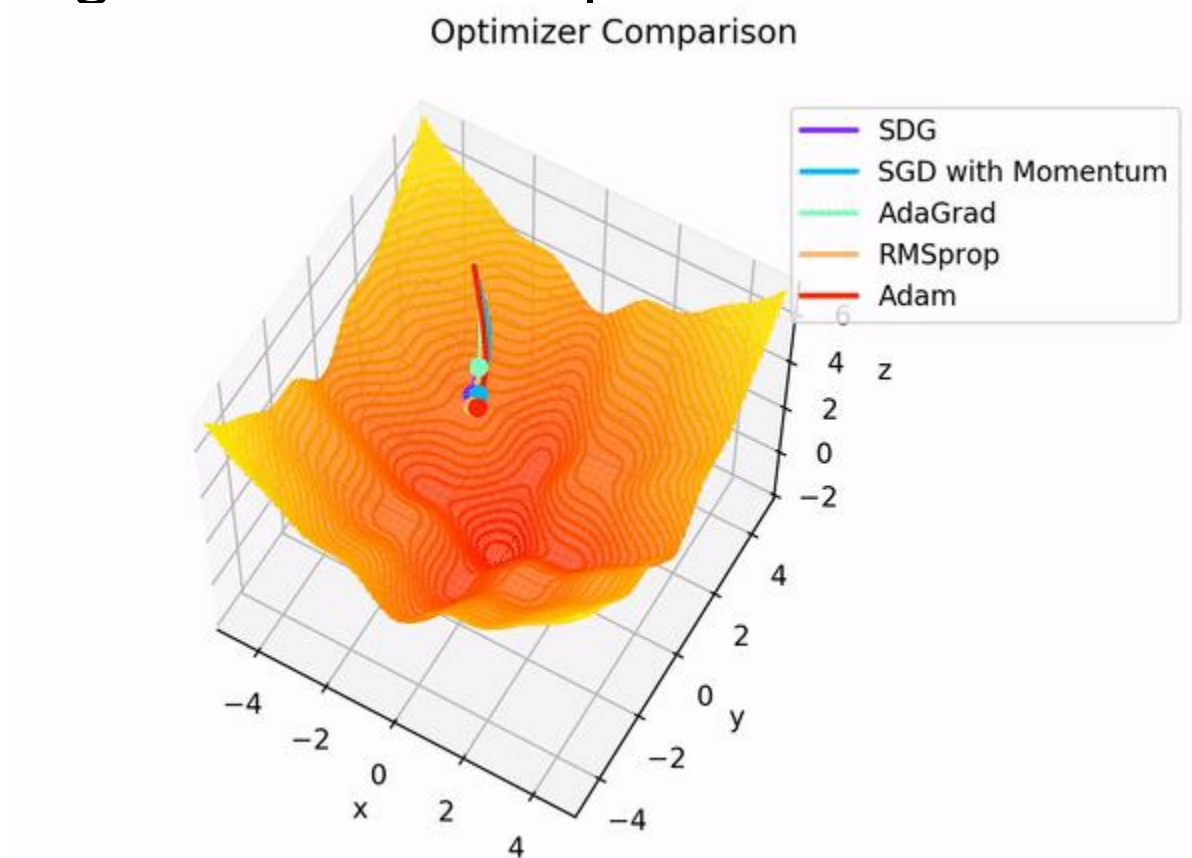
Learning Algorithms – examples



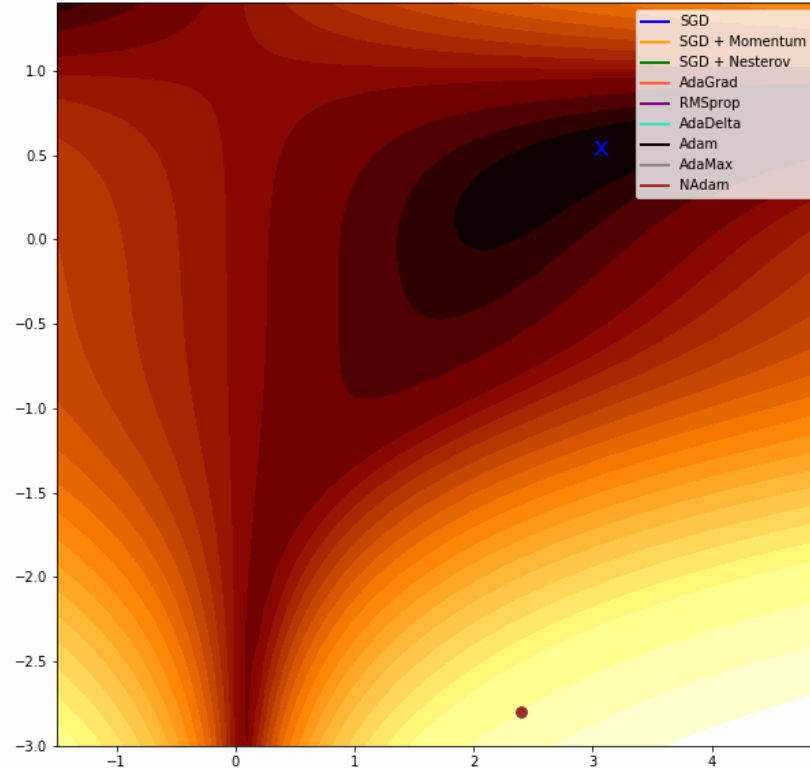
Learning Algorithms – examples



Learning Algorithms – examples



Learning Algorithms – examples



Learning Algorithms – another approaches

9. Second order methods – e.g. Newton's method, Levenberg-Marquardt algorithm, Nelder-Mead method, etc.

- may be faster and more precise than first order methods, but:
- they are not usually used for ANN training, because of:
 - high dimensionality of data and parameter sets
 - very complicated criterion functions
 - need to compute or approximate of inverse Hessian (inverse matrix of second order partial derivatives)
 - very memory and computationally demanding for these problems

10. Zero order methods – e.g. meta-heuristic/heuristic methods (evolution algorithms, etc.)

- not widely used – first order methods are enough for given problem and they are computable in a finite time + first order methods are mathematically proved and converge
- generally can be used, especially in problems, where the first derivation or gradient can't be computed (loss function is not differentiable) or its computation is inadequately demanding

Learning Algorithms – Summary

- use mini-batch modifications of learning algorithms (if it is possible)
- use at least momentum or NAG – nowadays, Adam is standard
- use at least Adam if you don't want problems with the initial learning rate setting
- use learning rate schedules (even for β_1, β_2 in Adam)

Loss Functions

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization

Loss functions

- Regression

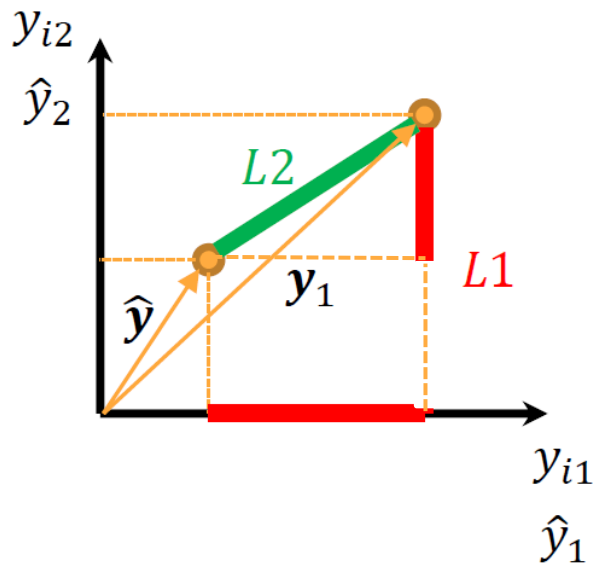
- L1 norm of the difference (Mean Absolute Error)

$$L_i = \|\hat{y} - y_i\|_1 = |\hat{y} - y_i|$$

- L2 squared norm (Euclidean norm, Mean Squared Error)

$$L_i = \|\hat{y} - y_i\|_2^2 = \sqrt{(\hat{y} - y_i)^2}$$

$$L_1^{(L1)} = \|\hat{\mathbf{y}} - \mathbf{y}_1\|_1 = \sum_d |\hat{y}_d - y_{1d}| = |\hat{y}_1 - y_{11}| + |\hat{y}_2 - y_{12}|$$



$$L_1^{(L2)} = \|\hat{\mathbf{y}} - \mathbf{y}_i\|_2^2 = \sqrt{(\hat{y}_1 - y_{11})^2 + (\hat{y}_2 - y_{12})^2}$$

Loss functions

Total loss – for epoch/mini-batch $L = \sum_{i=1}^N L_i$

- Classification

- Multi-class Weighted Cross-entropy (WCE) – for *one-hot encoded* classes – if sum of $\beta_c = 1$, balanced – classification problem

$$L_i = \sum_{c=1}^C \beta_c y_{i,c} \log(\hat{y}_{i,c})$$

C – number of classes

c – class index

β_c – weight of class

$y_{i,c}$ – true class

$\hat{y}_{i,c}$ – predicted class (with Softmax activation function)

- Binary class Tversky – for segmentation problem

$$T = \frac{\sum_{i=1}^N y_{i,1} \hat{y}_{i,1}}{\sum_{i=1}^N y_{i,1} \hat{y}_{i,1} + \beta \sum_{i=1}^N y_{i,1} \hat{y}_{i,0} + \alpha \sum_{i=1}^N y_{i,0} \hat{y}_{i,1}}$$

TP
 FN
 FP

α, β – weights of trade-off between FP and FN

if $\alpha = \beta = 0.5$ – Dice coefficient (F1 score)

if $\alpha = \beta = 1.0$ – Tanimoto coefficient (Jaccard)

if $\alpha + \beta = 1.0$ – higher $\beta \rightarrow$ higher weight for FNs $\rightarrow$ higher TPR

Regularization

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization

Regularization

- L1 and L2 regularization (Weight Decay) – see **Linear Regression topics**
 - regularization of weights of neural network during weights update
 - addition to loss function

$$\mathbf{W}^{(it+1)} = \mathbf{W}^{(it)} - \mu \nabla_{\mathbf{W}^{(it)}} (\mathbf{L} + \lambda \|\mathbf{W}^{(it)}\|_2^2)$$

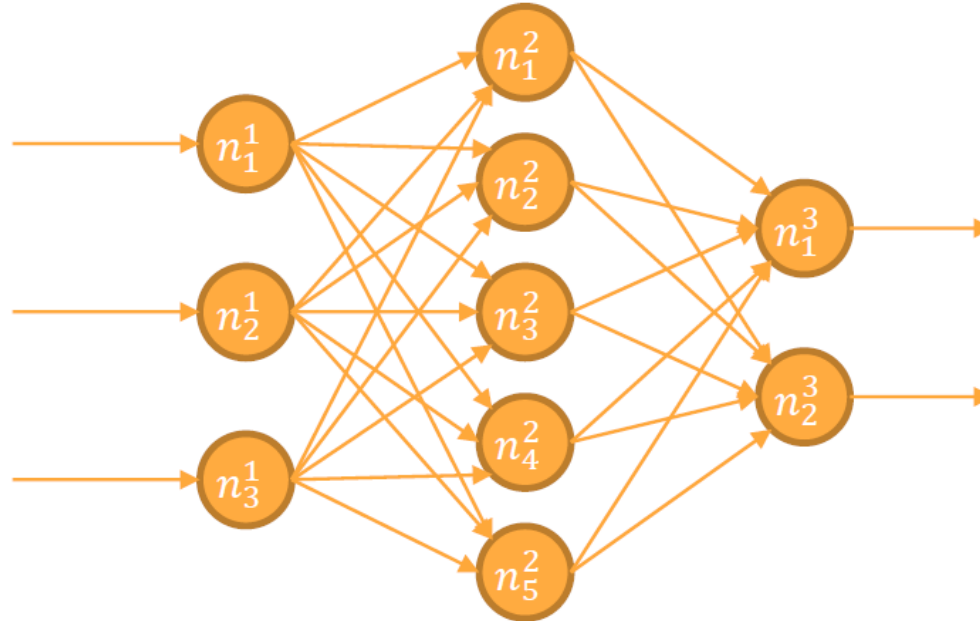
λ – regularization hyperparameter

Regularization – DropOut

- Motivation
 - We know, that combining of more different models improves the overall performance.
 - Also we know, that one complex model easily overfits and has high computational demands.
 - But, in the ANNs, we need to create many different architectures with unknown different hyperparameters, and ideally, we should train each model on different subset of training data. This is very complicated, time consuming and computationally demanding approach.

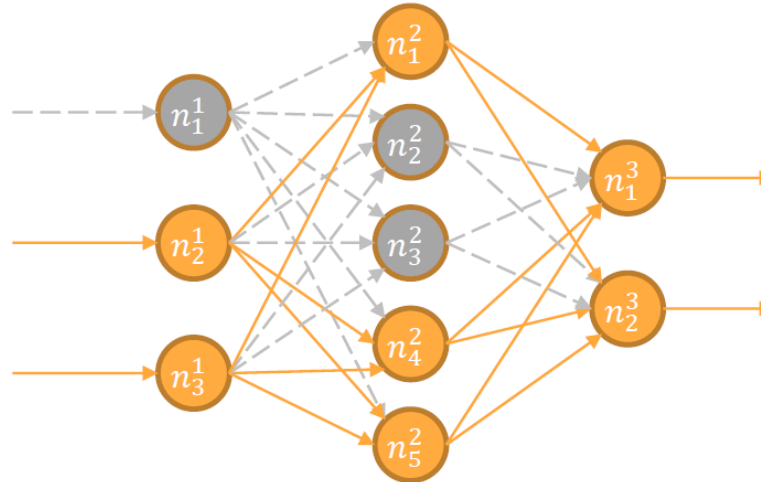
Regularization – DropOut

- What is it and how it works:
 - Full Neural Network



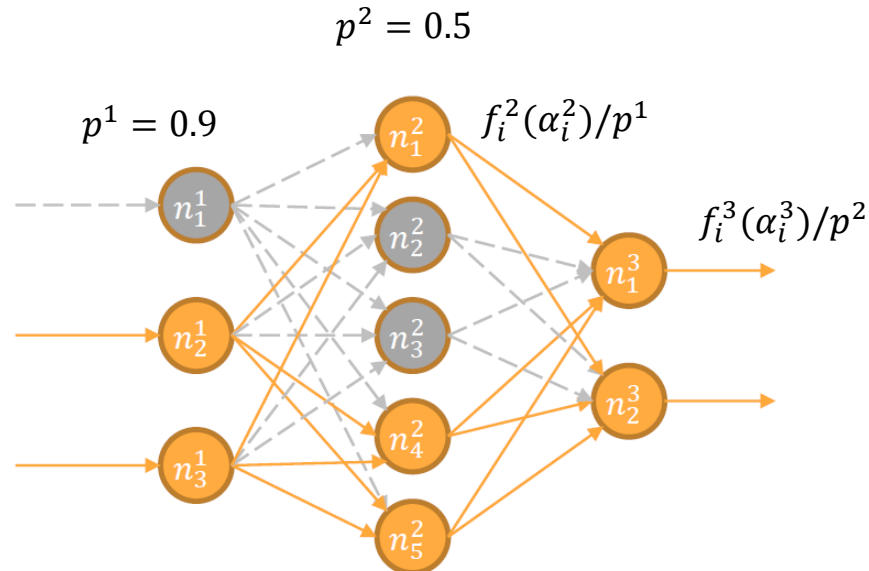
Regularization – DropOut

- What is it and how it works – Forward pass:
 - In each training epoch/mini-batch sample it randomly disables (“drops out”) some neurons in each layer in NN with all their input and output connections. DropOut state is given by Bernoulli random variable with given probability p for each neuron (1 = keep neuron, 0 = disable neuron, higher p means higher probability to be kept).
 - Combination in the last (output) layer – output neurons can’t be disabled – they define output class!



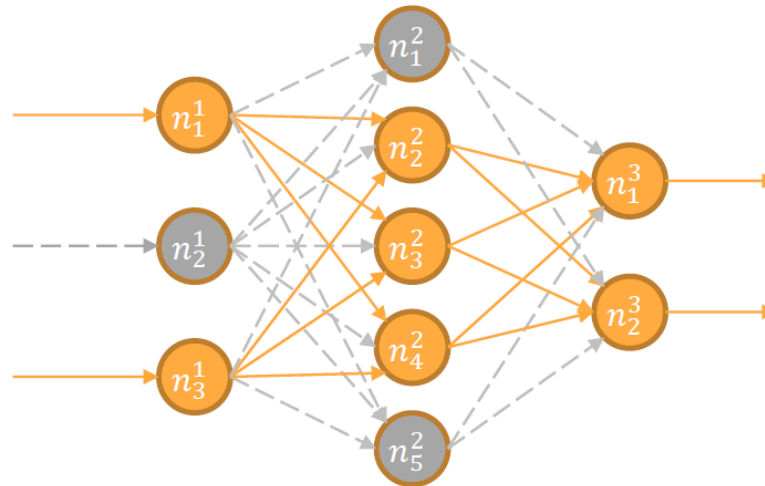
Regularization – DropOut

- What is it and how it works – Forward pass:
 - For this epoch/minibatch keep these neurons disabled
 - Output of each neuron in the next layer is divided by parameter p from previous layer – conservation of energy – especially important in deep architectures



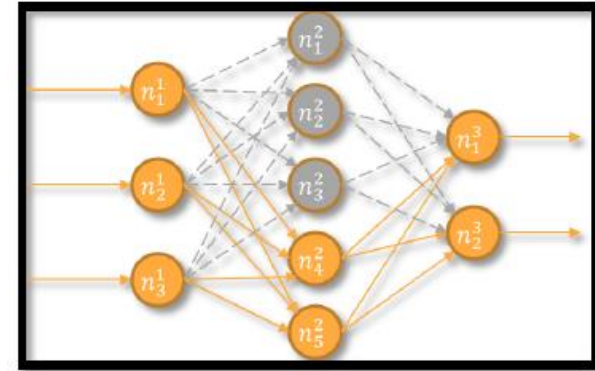
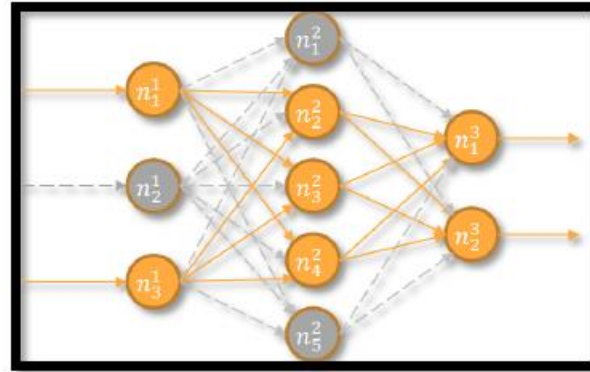
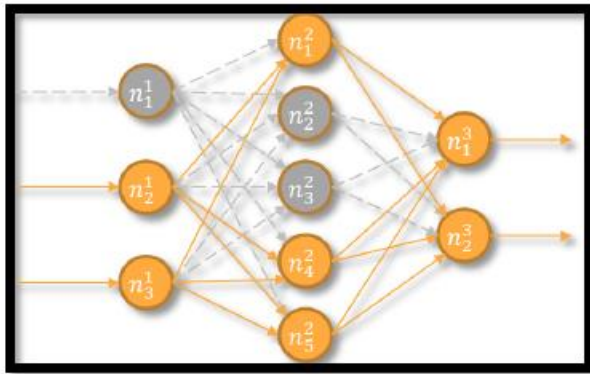
Regularization – DropOut

- What is it and how it works – Backward pass:
 - Compute loss and gradient only for neurons, that were not disabled
 - Update only weights of active neurons
 - Start new epoch/mini-batch: reactivate disabled neurons and continue training with a new realization of DropOut



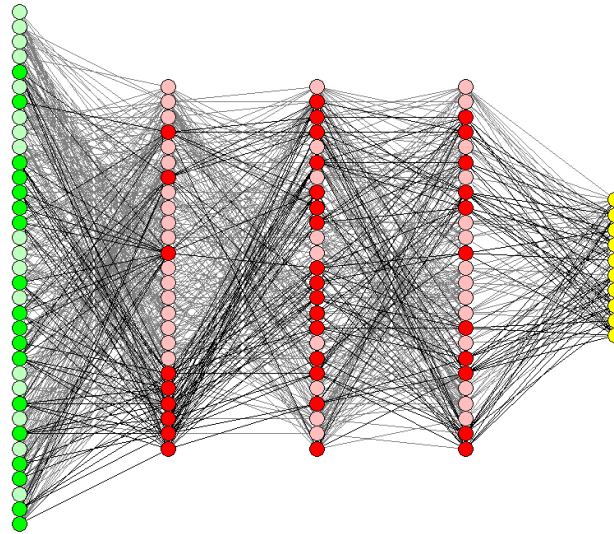
Regularization – DropOut

- What is it and how it works:
 - in each epoch/mini-batch a different small “thinned” sub-net is trained (regularization by reducing the complexity/size of model) – but they share their parameters (weights)



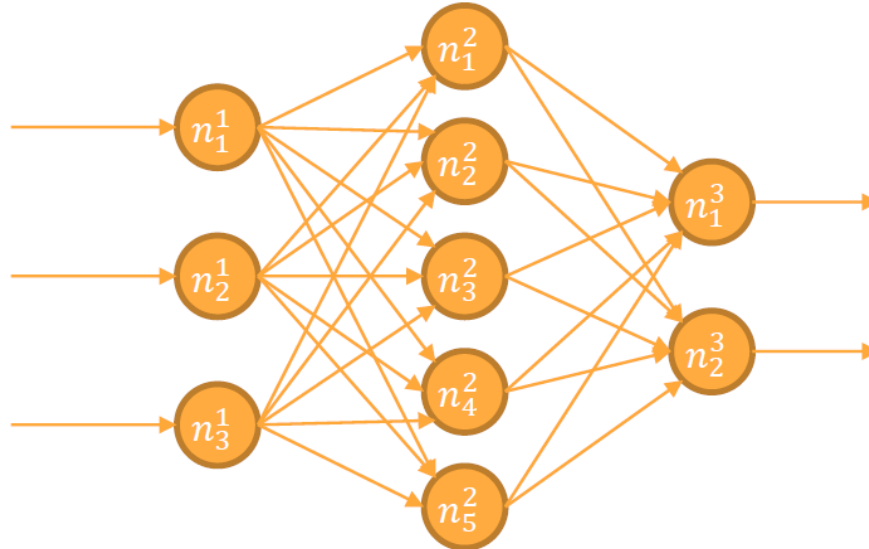
Regularization – DropOut

- What is it and how it works:
 - in each epoch/mini-batch a different small “thinned” sub-net is trained (regularization by reducing the complexity/size of model) – but they share their parameters (weights)



Regularization – DropOut

- What is it and how it works – Prediction/testing phase:
 - Combine all sub-nets together by activating all neurons!
 - Because the division of neural outputs by p was used during training, no other modification is needed – otherwise multiplication must be used in prediction phase!



Regularization – DropOut – Summary

- prevents overfitting
- approximation of “bagging” technique
- easy and simple
- fast to compute
- full trained NN approximately combines exponentially many different NN architectures – 2^N of possible sub-nets, where N is total number of neurons in the full NN
- thanks to weight sharing – still $O(n^2)$ parameters only
- can be (and should be) used together with L1/L2 regularization
- usual setting of p : for input layer near to 1.0, for hidden layers 0.5, for output layers 1.0!
- p can be also individually tuned
- could be ineffective if there are highly correlated features

Regularization – Batch Normalization

- normalization step that fixes means and variances of each layer's input
- changing normalization for each mini-batch and during training (because of changes of distribution of activations with weight changes and data changes)

$$D = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$$

1. compute mean and variance of mini-batch B for each feature/activation d separately – each mini-batch contains m samples of training data and store it for whole dataset

$$\mu_{B_d} = \frac{1}{m} \sum_{i=1}^m x_d^{(i)} \quad \sigma_{B_d}^2 = \frac{1}{m} \sum_{i=1}^m (x_d^{(i)} - \mu_{B_d})^2 \quad \begin{aligned} \mu_{D_d} &= \alpha \mu_{D_d} + (1 - \alpha) \mu_{B_d} \\ \sigma_{D_d}^2 &= \alpha \sigma_{D_d}^2 + (1 - \alpha) \sigma_{B_d}^2 \end{aligned} \quad \alpha - \text{exponential decay}$$

2. normalize mini-batch B according to these means and variances

$$\tilde{x}_d^{(i)} = \frac{x_d^{(i)} - \mu_{B_d}}{\sqrt{\sigma_{B_d}^2 + \epsilon}} \quad \epsilon - \text{arbitrary small constant}$$

Regularization – Batch Normalization

3. transforms normalized data $\tilde{x}_d^{(i)}$ by learnable parameters scale γ_d and shift β_d :

$$\tilde{h}_d^{(i)} = \gamma_d \tilde{x}_d^{(i)} + \beta_d$$
4. pass the batch normalized data $\tilde{h}_d$ to next network layers
5. by chain rule (BN is differentiable) train parameters scale γ_d and shift β_d
6. repeat 1. – 6. during training
7. AFTER TRAINING: calculate BN transformation parameters γ_d^{pred} and β_d^{pred} from point 3., that will be used in prediction phase as:

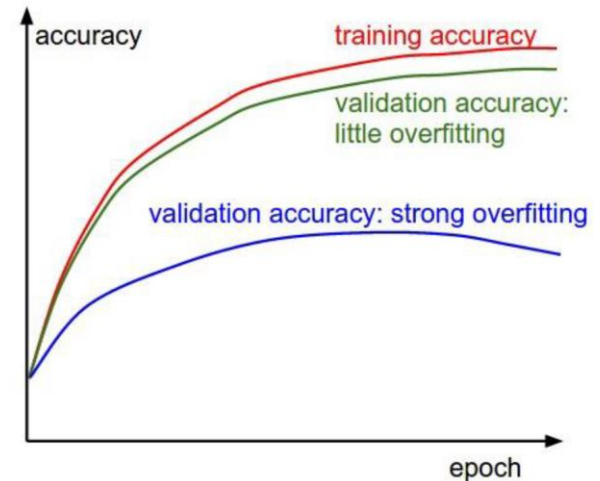
$$\tilde{h}_d^{pred} = \underbrace{\gamma_d^{pred}}_{\gamma_d} \underbrace{\frac{\gamma_d}{\sqrt{\sigma_{D_d}^2 + \epsilon}}}_{\beta_d - \frac{\gamma_d \mu_{D_d}}{\sqrt{\sigma_{D_d}^2 + \epsilon}}} x_d^{pred} + \underbrace{\left(\beta_d - \frac{\gamma_d \mu_{D_d}}{\sqrt{\sigma_{D_d}^2 + \epsilon}} \right)}_{\beta_d^{pred}}$$

Regularization – Batch Normalization – Summary

- prevents overfitting
- nowadays is shown that BN performs better than DropOut (no need of DropOut and sometimes together with DropOut performs worse)
- single linear transform in prediction phase (no computational complexity)
- reduces dependency on initial LR and weights
- enables higher LR
- enables lower L1/L2 regularization
- enables faster LR decay
- reduces the need of using pre-processing of the data
- speeds up training process
- needs appropriate size of the mini-batch – may be problem with a memory in huge models and/or size or dimensions of input data – not working with too small mini-batches
- dependent on mini-batch size – should be tuned

Regularization – another, but basic approaches

- limit the number of learning iterations/epochs
 - manually before training
 - manually during training
 - based on the number of validation checks
 - based on shape of learning curve (loss function changes during training)
 - comparing the shape of learning curves on training and validation dataset or the train/validation error gap (bias-variance trade-off)
- limit the size of model
 - number of hidden layers
 - number of neurons
 - “bottlenecks”



Tricks and Tips

- Basics
 - principle of neural nets
 - single neuron
 - principle
 - activation functions
 - derivation
 - general neural net
 - gradient backpropagation
- Learning Algorithms
- Loss Functions
- Regularization
- Tricks and Tips
 - weights initialization
 - learning rate schedules
 - hyperparameter optimization

Tricks and Tips

- initialization of weights

- non-zero random (Gaussian distribution with zero mean and “some” finite variance) for weights and zeros for biases, but:
- for tanh activation
 - too small initial weights → only linear activation → no benefit of multi-layer net
 - too high initial weights → saturated activation → zero gradient
 - so, we need the same variance on outputs of neurons as on their inputs:

$$\begin{aligned}
 \text{var}(y) &= \text{var}(w_1x_1 + w_2x_2 + \dots + w_Nx_N + b) \\
 \text{var}(w_ix_i) &= \text{var}(w_1x_1 + w_2x_2 + \dots + w_Nx_N + b) = E(x_i)^2 \text{var}(w_i) + E(w_i)^2 \text{var}(x_i) + \text{var}(w_i)\text{var}(x_i) \\
 \text{var}(y) &= \text{var}(w_1)\text{var}(x_1) + \text{var}(w_2)\text{var}(x_2) + \dots + \text{var}(w_N)\text{var}(x_N)
 \end{aligned}$$

$$\begin{aligned}
 \left. \begin{aligned} \text{var}(y) &= N \text{var}(w_i)\text{var}(x_i) \\ \text{var}(y) &= \text{var}(x_i) \end{aligned} \right\} N \text{var}(w_i) = 1 \quad \Rightarrow \quad \text{var}(w_i) = \frac{1}{N} \quad \Rightarrow \quad w_i = \text{randn} \cdot \sqrt{\frac{1}{N}}
 \end{aligned}$$

Tricks and Tips

- initialization of weights

- **Xavier** Glorot (2010) **normalized initialization** for tanh activation functions:

$$W \sim U \left[-\frac{\sqrt{6}}{\sqrt{N_{input} + N_{output}}}, \frac{\sqrt{6}}{\sqrt{N_{input} + N_{output}}} \right]$$

← solving problem
in forward and
backward pass

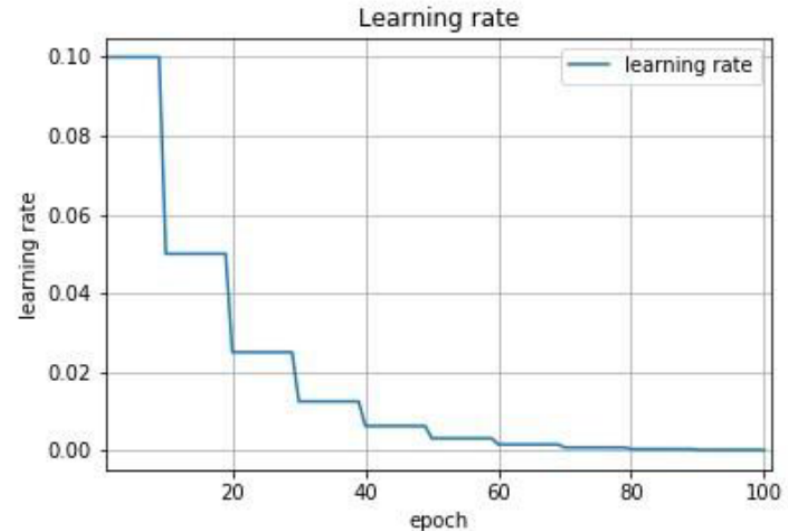
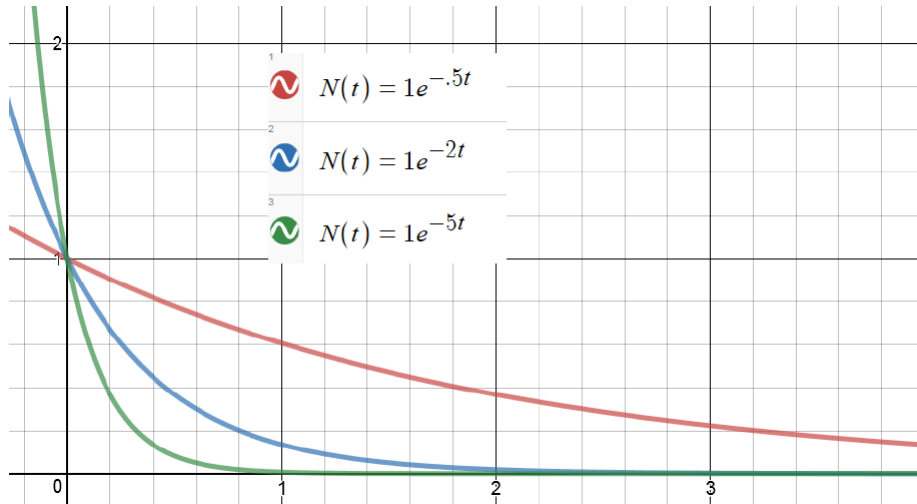
- Kaiming **He** (2015) **initialization** for ReLU activation functions:

$$W \sim N \left[-\frac{\sqrt{2}}{\sqrt{N_{input}}}, \frac{\sqrt{2}}{\sqrt{N_{input}}} \right]$$

Tricks and Tips

- learning rate schedules
 - step decay
 - exponential decay
 - 1/t decay $\mu = \mu_0 / (1 + kt)$

μ_0 – hyperparameter – initial LR
 k – hyperparameter – speed of decay
 t – iteration step/epoch



Tricks and Tips

- hyperparameter optimization
 - many hyperparameters that influence learning process and final validation performance ($\mu_0, \lambda, p, \alpha, k, \text{minibatch size}, \beta_1, \beta_2, \text{etc.}$)
 - each change of hyperparameter requires retraining of NN



grid search – brute-force search may take several days/weeks/months



random search – better than grid search

- can be improved by coarse to fine search
- be careful on borders
- may performs better than other if search intervals are chosen appropriately



evolution algorithms – see MPC-EAL



Bayesian optimization – [Hyperopt](#), [Spearmin](#), [SMAC](#)

